Plotter Class Documentation

Crystallographic Plotting Library Module

Detailed Class Documentation

November 28, 2025

Contents

1 Overview **2**
- 1.1 Key Features . . . 2
- 1.2 Attribute Categories . . . 2

2 Class Initialization **2**
- 2.1 Method: `__init__()` . . . 2
  - 2.1.1 Description . . . 2
  - 2.1.2 Input . . . 2
  - 2.1.3 Output . . . 2
  - 2.1.4 Usage Example . . . 3

3 Core Methods **3**
- 3.1 Method: `setAttributes(**kwargs)` . . . 3
  - 3.1.1 Description . . . 3
  - 3.1.2 Input . . . 3
  - 3.1.3 Output . . . 3
  - 3.1.4 Usage Example . . . 3
- 3.2 Method: `plotProj(**kwargs)` . . . 4
  - 3.2.1 Description . . . 4
  - 3.2.2 Input . . . 4
  - 3.2.3 Output . . . 4
  - 3.2.4 Usage Examples . . . 4
- 3.3 Method: `getScales(vmcbar, numticks=None, ticks=None, tickslabels=None, geq=False, leq=False, cmapbins=100, cmapbinsmult=None)` . . . 5
  - 3.3.1 Description . . . 5
  - 3.3.2 Input . . . 5
  - 3.3.3 Output . . . 5
  - 3.3.4 Usage Examples . . . 5
- 3.4 Method: `getFigparam(fontsize=None, save=False, phase='A,', figsize=None, **kwargs)` . . . 6
  - 3.4.1 Description . . . 6
  - 3.4.2 Input . . . 6
  - 3.4.3 Output . . . 6
  - 3.4.4 Usage Examples . . . 7
- 3.5 Method: `figsave(**kwargs)` . . . 7
  - 3.5.1 Description . . . 7
  - 3.5.2 Input . . . 7

3.5.3 Output . . . 7
3.5.4 Usage Examples . . . 7
3.6 Method: `figsaveproc(fname, **kwargs)` . . . 8
3.6.1 Description . . . 8
3.6.2 Input . . . 8
3.6.3 Output . . . 8
3.6.4 Usage Example . . . 8

4 Complete Usage Examples **9**
4.1 Basic Pole Figure . . . 9
4.2 Density Contour Plot . . . 9
4.3 Advanced Multi-Panel Figure . . . 9

5 Standalone Utility Functions **10**
5.1 Function: `get_cmap(colors, nbins=1000, name='my_cmap')` . . . 10
5.1.1 Description . . . 10
5.1.2 Input . . . 10
5.1.3 Output . . . 10
5.1.4 Usage Example . . . 10
5.2 Function: `get_colors(values, cmap, vmin=None, vmax=None, cmin=0, cmax=None, to255=True, nancolor=[0, 0, 0, 1])` . . . 11
5.2.1 Description . . . 11
5.2.2 Input . . . 11
5.2.3 Output . . . 11
5.2.4 Usage Example . . . 11
5.3 Function: `shiftedColorMap(cmap, start=0, midpoint=0.5, stop=1.0, name='shiftedcmap')` 1
5.3.1 Description . . . 12
5.3.2 Input . . . 12
5.3.3 Output . . . 12
5.3.4 Usage Example . . . 12

6 Best Practices **13**
6.1 Figure Quality . . . 13
6.2 Color Scales . . . 13
6.3 Projections . . . 13
6.4 Performance . . . 13

7 Mathematical Background **14**
7.1 Stereographic Projection . . . 14
7.2 Color Scale Normalization . . . 14
7.3 Colormap Shifting . . . 14

8 Troubleshooting **14**
8.1 Common Issues . . . 14
8.1.1 Issue: Figures not displaying . . . 14
8.1.2 Issue: Low resolution output . . . 14
8.1.3 Issue: Colormap appears banded . . . 14
8.1.4 Issue: Cropping not working . . . 15

9 References **15**

1 Overview

The `plotter` class provides a comprehensive framework for creating publication-quality crystallographic plots including stereographic projections, pole figures, and orientation density maps.

1.1 Key Features

- Stereographic projections (Schmidt/Wulff nets)
- Pole figures with density contours
- Scatter plots with crystallographic indexing
- Interactive data annotations
- Customizable colormaps and color scales
- Support for multiple crystal systems and symmetry operations

1.2 Attribute Categories

- **Colorbar**: Color scale display and formatting
- **Figure**: Overall figure layout and titles
- **Colormap**: Data-to-color mapping and contours
- **Projection**: Stereographic projection type and settings
- **Axis**: Plot styling, ticks, labels
- **Crystal**: Crystal structure and symmetry operations
- **Save**: File output settings
- **Scatter**: Scatter plot data and styling
- **Interactive**: Mouse-over annotations

2 Class Initialization

2.1 Method: `__init__()`

2.1.1 Description

Initialize the plotter with default attributes for all plotting categories. All attributes can be modified using the `setAttributes()` method.

2.1.2 Input

- None

2.1.3 Output

- None (initializes plotter instance)

2.1.4 Usage Example

```python
import numpy as np
from plotlib import plotter

# Create plotter instance
p = plotter()

# Access default values
print("Default colormap:", p.cmap)
print("Default projection sphere:", p.sphere)

# Modify attributes
p.setAttributes(cmap='viridis', sphere='half')
```

Listing 1: Basic initialization

3 Core Methods

3.1 Method: `setAttributes(**kwargs)`

3.1.1 Description

Set or update plotter attributes dynamically. This method allows flexible configuration of any plotter attribute without needing to know the internal structure.

3.1.2 Input

- `**kwargs`: keyword arguments – Any attribute name and value pairs

3.1.3 Output

- None (updates instance attributes)

3.1.4 Usage Example

```python
p = plotter()

# Set single attribute
p.setAttributes(cmap='viridis')

# Set multiple attributes
p.setAttributes(
    sphere='half',
    ProjType='equalarea',
    figsize=(8, 8),
    dirs=[[1,0,0], [1,1,0]],
    cmap='jet'
)

# Verify changes
print("Sphere:", p.sphere)
print("Colormap:", p.cmap)
```

Listing 2: Setting attributes

3.2 Method: `plotProj(**kwargs)`

3.2.1 Description

Create a stereographic projection (pole figure) with appropriate grid. Sets up the figure, axes, and projection type (equal-area Schmidt net or equal-angle Wulff net). Handles full sphere, hemisphere, or standard stereographic triangle projections.

3.2.2 Input

- `ProjType`: str – `'equalarea'` (Schmidt) or `'equalangle'` (Wulff)
- `sphere`: str – `'full'`, `'half'`, or `'triangle'`
- `figsize`: tuple – Figure size (width, height) in inches
- `stereogrid`: bool – Display stereographic grid
- `stereoresolution`: int – Grid resolution
- `stereomesh`: bool – Display mesh
- `ax`: matplotlib axis – Existing axis to plot on (optional)
- `fig`: matplotlib figure – Existing figure (optional)

3.2.3 Output

- None (creates/modifies `self.fig` and `self.ax`)

3.2.4 Usage Examples

Example 1: Equal-area projection, upper hemisphere

```python
import matplotlib.pyplot as plt

# Equal-area projection, upper hemisphere
p = plotter()
p.plotProj(ProjType='equalarea', sphere='half', figsize=(6, 6))
plt.show()
```

Listing 3: Equal-area hemisphere projection

Example 2: Equal-angle projection, full sphere

```python
# Equal-angle projection, full sphere
p2 = plotter()
p2.plotProj(ProjType='equalangle', sphere='full')
plt.show()
```

Listing 4: Equal-angle full sphere projection

Example 3: Standard stereographic triangle

```python
# Standard stereographic triangle (for cubic)
p3 = plotter()
p3.plotProj(
    ProjType='equalarea',
    sphere='triangle',
    stereogrid=True,
    stereoresolution=10
)
```

```
plt.show()
```

Listing 5: Stereographic triangle for cubic systems

3.3 Method: `getScales(vmcbar, numticks=None, ticks=None, tickslabels=None, geq=False, leq=False, cmapbins=100, cmapbinsmult=None)`

3.3.1 Description

Generate color scale parameters including ticks, labels, and colormap. Creates a gradient colormap from white → blue → green → yellow → dark red with customizable tick positions and labels.

3.3.2 Input

- `vmcbar`: list/array [min, max] – Value range for colorbar
- `numticks`: int – Number of tick marks (optional, auto-computed if None)
- `ticks`: array – Explicit tick positions (optional)
- `tickslabels`: list of str – Custom tick labels (optional)
- `geq`: bool – Add $\geq$ symbol to maximum tick (default: False)
- `leq`: bool – Add $\leq$ symbol to minimum tick (default: False)
- `cmapbins`: int – Number of colormap bins (default: 100)
- `cmapbinsmult`: int – Multiplier for bins based on ticks (optional)

3.3.3 Output

Dictionary containing:

- `'tickslabels'`: list of str – Formatted tick labels
- `'vm'`: list [min, max] – Colormap value range
- `'vmbar'`: list [min, max] – Colorbar value range
- `'cmap'`: matplotlib colormap – Generated colormap
- `'ticks'`: array – Tick positions

3.3.4 Usage Examples

Example 1: Basic scale

```
p = plotter()

# Basic scale from 0 to 10
scales = p.getScales(vmcbar=[0, 10])
print("Tick labels:", scales['tickslabels'])
print("Colormap range:", scales['vm'])
```

Listing 6: Basic color scale

Example 2: Custom ticks with inequality symbols

```python
import numpy as np

scales2 = p.getScales(
    vmcbar=[0, 100],
    ticks=np.array([0, 25, 50, 75, 100]),
    geq=True,  # Add >= to maximum
    leq=True   # Add <= to minimum
)
print("Labels:", scales2['tickslabels'])
# Output: ['<=0', '25', '50', '75', '>=100']
```

Listing 7: Custom ticks with inequality symbols

Example 3: High-resolution colormap

```python
scales3 = p.getScales(
    vmcbar=[0, 1],
    cmapbins=1000  # Smooth gradient
)
```

Listing 8: High-resolution smooth colormap

3.4 Method: `getFigparam(fontsize=None, save=False, phase='A,', figsize=None, **kwargs)`

3.4.1 Description

Get figure parameter dictionary for saving high-quality figures.

3.4.2 Input

- `fontsize`: float – Font size for text (optional)
- `save`: bool – If True, use smaller figure size (default: False)
- `phase`: str – Phase identifier (`'A'` or other) affects default size
- `figsize`: tuple – Custom figure size (width, height) in inches
- `**kwargs`: Additional parameters to override defaults

3.4.3 Output

Dictionary containing figure parameters for `plt.savefig()`:

- `'dpi'`: int – Resolution (default: 300)
- `'facecolor'`: str – Background color
- `'edgecolor'`: str – Edge color
- `'orientation'`: str – `'portrait'` or `'landscape'`
- `'format'`: str – Image format
- `'transparent'`: bool – Transparent background
- `'bbox_inches'`: str – Bounding box setting
- `'pad_inches'`: float – Padding around figure

3.4.4 Usage Examples

Example 1: Default parameters

```
p = plotter()

# Get default parameters
params = p.getFigparam()
print("DPI:", params['dpi'])
```

Listing 9: Get default figure parameters

Example 2: High-resolution PDF

```
params_pdf = p.getFigparam(
    save=True,
    format='pdf',
    dpi=600
)
```

Listing 10: High-resolution PDF parameters

Example 3: Use with savefig

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.plot([1, 2, 3], [1, 4, 9])
fig.savefig('output.png', **params)
```

Listing 11: Using parameters with matplotlib

3.5 Method: `figsave(**kwargs)`

3.5.1 Description

Save figure to file(s) with optional cropping. Supports multiple image formats and automatic cropping of whitespace. Can save to multiple formats simultaneously.

3.5.2 Input

- `fname`: str or list – Filename(s) to save
- `imformats`: list of str – Image formats [`'png'`, `'pdf'`, `'svg'`, etc.]
- `crop`: bool – Auto-crop whitespace (requires wand library)
- `figparam`: dict – Figure parameters (see `getFigparam`)

3.5.3 Output

- None (saves file(s) to disk)

3.5.4 Usage Examples

Example 1: Save single format

```
p = plotter()
p.plotProj(ProjType='equalarea')

# Save single format
p.figsave(fname='pole_figure.png')
```

Listing 12: Save to single format

Example 2: Save multiple formats

```
# Save multiple formats
p.figsave(
    fname='pole_figure.png',
    imformats=['png', 'pdf', 'svg']
)
# Creates: pole_figure.png, pole_figure.pdf, pole_figure.svg
```

Listing 13: Save to multiple formats simultaneously

Example 3: Save with cropping

```
# Save with cropping
p.figsave(
    fname='cropped_figure.png',
    crop=True
)
```

Listing 14: Save with automatic whitespace cropping

Example 4: High DPI save

```
# High DPI save
p.figparam['dpi'] = 600
p.figsave(fname='high_res.png')
```

Listing 15: High resolution output

3.6 Method: `figsaveproc(fname, **kwargs)`

3.6.1 Description

Process and save figure to file. Internal method called by `figsave()` to handle actual file writing.

3.6.2 Input

- `fname`: str – Filename to save
- `**kwargs`: Additional parameters to update

3.6.3 Output

- None (saves file to disk)

3.6.4 Usage Example

```
# Typically called internally by figsave()
# But can be used directly:
p = plotter()
p.plotProj()
p.figsaveproc('output.png')
```

Listing 16: Direct file saving (typically called internally)

4 Complete Usage Examples

4.1 Basic Pole Figure

```
import numpy as np
from plotlib import plotter

# Create plotter instance
p = plotter()

# Setup projection (equal-area, upper hemisphere)
p.plotProj(ProjType='equalarea', sphere='half')

# Plot crystal directions
p.setAttributes(
    dirs=[[1,0,0], [0,1,0], [0,0,1]],
    norms=[[1,1,1]]
)
p.plotDirsNorms()

# Save figure
p.figsave(fname='pole_figure.png')
```

Listing 17: Creating a basic pole figure

4.2 Density Contour Plot

```
import numpy as np
from plotlib import plotter

# Random orientations
orientations = np.random.randn(1000, 3, 3)

# Create density plot
p2 = plotter()
p2.setAttributes(oris=orientations, ProjType='equalarea')
p2.plotProj()
p2.plotColormap()
p2.plotColorbar()

# Save with multiple formats
p2.figsave(fname='density_plot.png', imformats=['png', 'pdf'])
```

Listing 18: Creating an orientation density plot

4.3 Advanced Multi-Panel Figure

```
import matplotlib.pyplot as plt
import numpy as np
from plotlib import plotter

# Create 2x2 subplot
fig, axes = plt.subplots(2, 2, figsize=(12, 12))

# Create multiple pole figures
for i, ax in enumerate(axes.flat):
```

```
    p = plotter()
    p.plotProj(
        ProjType='equalarea',
        sphere='half',
        ax=ax,
        fig=fig
    )

    # Different directions for each subplot
    dirs = [[i+1, 0, 0], [0, i+1, 0], [0, 0, i+1]]
    p.setAttributes(dirs=dirs)
    p.plotDirsNorms()

plt.tight_layout()
plt.savefig('multi_panel_figure.png', dpi=300)
```

Listing 19: Creating a multi-panel crystallographic figure

5 Standalone Utility Functions

5.1 Function: `get_cmap(colors, nbins=1000, name='my_cmap')`

5.1.1 Description

Create a custom colormap from a list of colors with smooth gradients.

5.1.2 Input

- `colors`: list of tuples/colors – Colors to interpolate between (RGB tuples or named colors)
- `nbins`: int – Number of discrete bins in colormap (default: 1000)
- `name`: str – Name for the colormap (default: `'my_cmap'`)

5.1.3 Output

- `cmap`: matplotlib.colors.LinearSegmentedColormap – Custom colormap

5.1.4 Usage Example

```
import matplotlib.pyplot as plt
import numpy as np
from plotlib import get_cmap

# Create colormap from white to red to black
colors = [(1, 1, 1), (1, 0, 0), (0, 0, 0)]
cmap = get_cmap(colors, nbins=256)

# Use in plot
data = np.random.rand(10, 10)
plt.imshow(data, cmap=cmap)
plt.colorbar()
plt.show()

# Multi-color gradient
colors_multi = [
    (0, 0, 1),      # Blue
```

```
    (0, 1, 1),        # Cyan
    (0, 1, 0),        # Green
    (1, 1, 0),        # Yellow
    (1, 0, 0)         # Red
]
cmap_rainbow = get_cmap(colors_multi, nbins=500)
```

Listing 20: Creating custom colormaps

5.2 Function: `get_colors(values, cmap, vmin=None, vmax=None, cmin=0, cmax=None, to255=True, nancolor=[0, 0, 0, 1])`

5.2.1 Description

Map data values to colors using a colormap.

5.2.2 Input

- `values`: numpy array – Data values to map to colors
- `cmap`: matplotlib colormap – Colormap to use
- `vmin`: float – Minimum data value (default: min of values)
- `vmax`: float – Maximum data value (default: max of values)
- `cmin`: int – Minimum colormap index (default: 0)
- `cmax`: int – Maximum colormap index (default: `cmap.N`)
- `to255`: bool – Scale colors to 0-255 range (default: True)
- `nancolor`: list – RGBA color for NaN values (default: black)

5.2.3 Output

- `Colors`: numpy array – Array of colors (RGBA), shape: (`values.shape`, 4)

5.2.4 Usage Example

```
import numpy as np
import matplotlib.pyplot as plt
from plotlib import get_colors

# Map values to colors
values = np.array([1, 5, 10, 15, 20])
cmap = plt.cm.viridis
colors = get_colors(values, cmap, vmin=0, vmax=20, to255=False)
print("Colors shape:", colors.shape)

# Use with scatter plot
x = np.random.rand(100)
y = np.random.rand(100)
z = np.random.rand(100) * 100
colors = get_colors(z, plt.cm.jet, to255=False)
plt.scatter(x, y, c=colors)
plt.show()

```

```
# Handle NaN values
values_with_nan = np.array([1, 2, np.nan, 4, 5])
colors_nan = get_colors(values_with_nan, plt.cm.coolwarm,
                        nancolor=[1, 1, 1, 1])  # White for NaN
```

Listing 21: Mapping values to colors

5.3 Function: `shiftedColorMap(cmap, start=0, midpoint=0.5, stop=1.0, name='shiftedcma`

5.3.1 Description

Shift the center of a colormap to a specific data value. Useful for data with asymmetric ranges (e.g., −15 to +5) where you want the colormap center (typically white or neutral color) at zero.

5.3.2 Input

- `cmap`: matplotlib colormap – The colormap to shift
- `start`: float – Offset from lowest point [0.0, midpoint] (default: 0.0)
- `midpoint`: float – New center position [0.0, 1.0] (default: 0.5)
  - Calculate as: $1 - \frac{v_{\max}}{v_{\max}+|v_{\min}|}$
- `stop`: float – Offset from highest point [midpoint, 1.0] (default: 1.0)
- `name`: str – Name for new colormap (default: `'shiftedcmap'`)

5.3.3 Output

- `newcmap`: matplotlib.colors.LinearSegmentedColormap – Shifted colormap

5.3.4 Usage Example

```
import matplotlib.pyplot as plt
import numpy as np
from plotlib import shiftedColorMap

# Data ranging from -15 to +5
data = np.random.randn(100, 100) * 10 - 5
vmin, vmax = -15, 5

# Calculate midpoint to center at zero
midpoint = 1 - vmax / (vmax + abs(vmin))
print(f"Midpoint: {midpoint:.3f}")  # 0.75

# Create shifted colormap
original_cmap = plt.cm.RdBu_r
shifted_cmap = shiftedColorMap(original_cmap, midpoint=midpoint)

# Plot with original vs shifted
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

im1 = ax1.imshow(data, cmap=original_cmap, vmin=vmin, vmax=vmax)
ax1.set_title('Original colormap')
plt.colorbar(im1, ax=ax1)

```

```
im2 = ax2.imshow(data, cmap=shifted_cmap, vmin=vmin, vmax=vmax)
ax2.set_title('Shifted colormap (zero = white)')
plt.colorbar(im2, ax=ax2)
plt.show()
```

Listing 22: Shifting colormap center for asymmetric data

6 Best Practices

6.1 Figure Quality

- Use DPI $\geq$ 300 for publication-quality figures
- Save in vector formats (PDF, SVG) when possible for scalability
- Enable cropping to remove unnecessary whitespace
- Use `tight_layout=True` for multi-panel figures

6.2 Color Scales

- Use perceptually uniform colormaps (viridis, plasma) for continuous data
- Use diverging colormaps (RdBu) for data centered around zero
- Add inequality symbols ($\geq$, $\leq$) for saturated color scales
- Ensure sufficient color resolution (`cmapbins` $\geq$ 100)

6.3 Projections

- Use equal-area (Schmidt) for texture analysis and density plots
- Use equal-angle (Wulff) for angular relationships
- Use hemisphere projections for single-phase materials
- Use stereographic triangle for cubic crystal systems

6.4 Performance

- Pre-compute orientation data before plotting
- Use lower resolution (`nump`) for preview, higher for final output
- Cache symmetry operations for repeated calculations
- Batch export multiple formats in single call to `figsave()`

7 Mathematical Background

7.1 Stereographic Projection

The stereographic projection maps points from a sphere to a plane. For a point $\mathbf{p} = (x, y, z)$ on the unit sphere, the stereographic projection from the north pole $(0, 0, 1)$ to the $z = 0$ plane is:

$$(X, Y) = \left(\frac{x}{1 - z}, \frac{y}{1 - z} \right) \tag{1}$$

For the equal-area (Schmidt) projection:

$$(X, Y) = \left(\frac{x}{\sqrt{1 + z}}, \frac{y}{\sqrt{1 + z}} \right) \tag{2}$$

7.2 Color Scale Normalization

Values are normalized to colormap indices using:

$$v_n = \frac{v - v_{\min}}{v_{\max} - v_{\min}} \cdot (c_{\max} - c_{\min}) + c_{\min} \tag{3}$$

where v is the data value, $v_{\min}$ and $v_{\max}$ are the data range, and $c_{\min}$ and $c_{\max}$ are the colormap index range.

7.3 Colormap Shifting

For asymmetric data ranges, the midpoint m should be calculated as:

$$m = 1 - \frac{v_{\max}}{v_{\max} + |v_{\min}|} \tag{4}$$

For example, with data ranging from -15 to $+5$:

$$m = 1 - \frac{5}{5 + 15} = 1 - \frac{5}{20} = 0.75 \tag{5}$$

8 Troubleshooting

8.1 Common Issues

8.1.1 Issue: Figures not displaying

- **Solution**: Call `plt.show()` after creating the plot
- **Alternative**: Use `withdraw=False` parameter

8.1.2 Issue: Low resolution output

- **Solution**: Increase DPI: `p.figparam['dpi'] = 600`
- **Alternative**: Save as vector format (PDF, SVG)

8.1.3 Issue: Colormap appears banded

- **Solution**: Increase `cmapbins`: `getScales(cmapbins=1000)`
- **Alternative**: Use `cmapbinsmult` for automatic scaling

8.1.4 Issue: Cropping not working

- **Solution**: Install wand library: `pip install Wand`
- **Note**: Cropping only works for PNG format

9 References

- Stereographic projection: Kocks, U. F., Tomé, C. N., & Wenk, H. R. (2000). *Texture and anisotropy.* Cambridge University Press.
- Crystallographic conventions: Bunge, H. J. (2013). *Texture analysis in materials science.* Elsevier.
- Color theory: Moreland, K. (2016). Why we use bad color maps and what you can do about it. *Electronic Imaging*, 2016(16), 1-6.